COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010: Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	10 / 13 / 26	**DATE SUBMITTED:**	10 / ? / 26
NAME:	Fabonan, Kurt Lorenz C. Santos, Paterno Miguel R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 12.1: Algorithmic Strategies

1. PROCEDURE OUTPUT

Strategy	Algorithm	Analysis
Recursion	Binary Search	In binary search, an assigned value called the midpoint is used. Then recursion when the value being searched is either less or more than the midpoint than the value being searched. The recursion calls itself to whether to search left or right in the array then calling it again when the value being searched is not present on the searched side.
Brute Force	Bubble Sorting	Bubble sorting swaps around elements in an array until it reaches an increasing order. This also makes it check every index multiple times until the entire array is completely in order.
Backtracking	Binary Search Tree Traversals	Binary Search Tree Traversals, although not backtracking themselves, use a backtracking style of movement in order to find the nodes of a tree. This is because the search traversal goes from left to right from the furthermost left, then when it hits

		a dead end, back and checks if there is another child, then it goes to that child, and repeats until it has reached the rightmost side of the tree.
Greedy	Selection Sorting	Selection sorting can have a greedy algorithm since it makes a locally optimal choice when it sorts. Since it searches for the smallest value then relocates the element to the desired index as it sorts.
Divide-and-Conquer	Merge and Quick Sorting	Merge sorting splits an array by two and then continuously splits the two arrays until one element is inside an array. Then it is compared repeatedly before it returns into one organized array. Quick sorting also has the same process, the only difference is that it is uses a pivot to separate the values into two arrays that will also use pivots to compare and split the arrays before it goes back into one organized array.

Table 12-1. Algorithmic Strategies and Examples

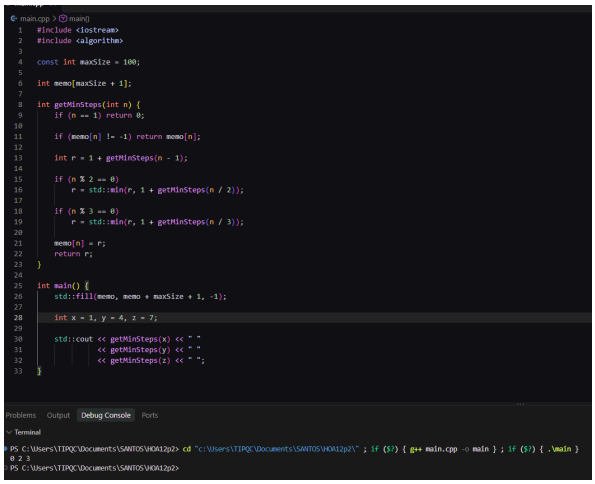
Screenshot	
Analysis	

Table 12-2. Memoization Implementation

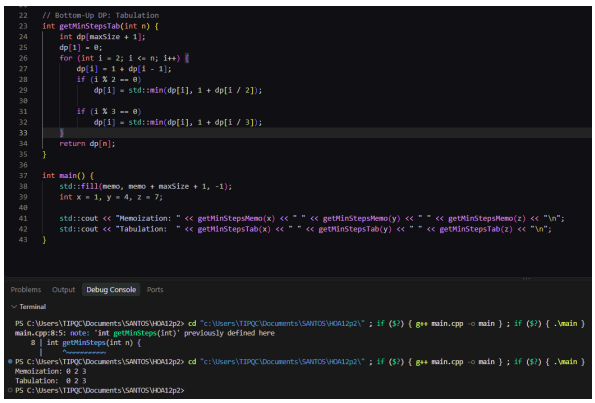
Screenshot	
Analysis	The code uses bottom

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies.

Problem Title: Count the number of paths in a matrix with a given cost to reach the destination cell
Description: Given an M × N integer matrix where each cell has a non-negative cost associated with it, count the number of paths to reach the last cell (M-1, N-1) of the matrix from its first cell (0, 0) such that the path has given cost. We can only move one unit right or one unit down from any cell, i.e., from cell (i, j), we can move to (i, j+1) or (i+1, j).

For example, consider the following 4 × 4 matrix where cost is 25. Two paths are having a cost of 25. A sample path has been given when the cost is 25.

4 7 1 6
6 7 3 9
3 8 1 2
7 1 7 3

The idea is to use recursion. The problem has optimal substructure. That means the problem can be broken down into smaller, simple "subproblems", which can further be divided into yet simpler, smaller subproblems until the solution becomes trivial.

Deliverables:

- Pseudocode

```
FUNCTION countPaths(matrix, row, col, remainingCost, M, N):

    IF row >= M OR col >= N:
        RETURN 0

    remainingCost = remainingCost - matrix[row][col]

    IF remainingCost < 0:
        RETURN 0

    IF row == M-1 AND col == N-1:
        IF remainingCost == 0:
            RETURN 1
        ELSE:
            RETURN 0

    rightPaths = countPaths(
        matrix, row, col+1, remainingCost, M, N
    )

    downPaths = countPaths(
        matrix, row+1, col, remainingCost, M, N
    )

    RETURN rightPaths + downPaths
```

```
MAIN:
    Read M, N
    Read matrix
    Read targetCost

    result = countPaths(matrix, 0, 0, targetCost, M, N)

    Print result
```

- Working C++ Code

```
#include <iostream>
#include <vector>

int M, N;
std::vector<std::vector<int>> matrix;
std::vector<std::vector<std::vector<int>>> memo;

int countPaths(int row, int col, int remainingCost) {
    // Outside the matrix or impossible remaining cost
    if (row >= M || col >= N || remainingCost < 0) {
        return 0;
    }

    // Include the current cell's cost
    remainingCost -= matrix[row][col];

    if (remainingCost < 0) {
        return 0;
    }

    // Destination reached
    if (row == M - 1 && col == N - 1) {
        return (remainingCost == 0) ? 1 : 0;
    }

    // Return a previously calculated result
    int &answer = memo[row][col][remainingCost];
    if (answer != -1) {
        return answer;
    }

    // Explore moving right and moving down
    int rightPaths = countPaths(
```

```
        row, col + 1, remainingCost
    );

    int downPaths = countPaths(
        row + 1, col, remainingCost
    );

    answer = rightPaths + downPaths;
    return answer;
}

int main() {
    std::cout << "Enter rows and columns: ";
    std::cin >> M >> N;

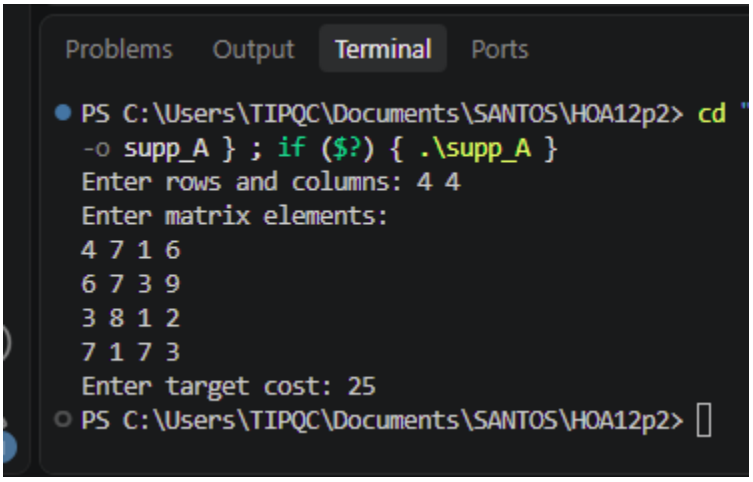
    matrix.resize(M, std::vector<int>(N));

    std::cout << "Enter matrix elements:\n";
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            std::cin >> matrix[i][j];
        }
    }

    int targetCost;
    std::cout << "Enter target cost: ";
    std::cin >> targetCost;

    if (targetCost < 0) {
        std::cout << "Number of valid paths: 0\n";
        return 0;
    }
}
```

- Analysis of working code
- Screenshots of demonstration



3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.